# North South University

## Department of Electrical & Computer Engineering

## Lab Report

**Experiment No:** 01

**Experiment Title:** Design of a 2-bit Logic Unit

**Course Code:** CSE332L

**Section:** 07

**Course Name:** Computer Organization and Architecture Lab

**Lab Group #:** 01

**Written by:** Rowful Ahasan

**Date of Experiment:** 29-01-2026

**Date of Submission:** 05-02-2026

| Group Member's ID | Group Member's Name |
|---|---|
| 2411269042 | Rowful Ahasan |
| 2413893642 | Rafiqul Islam |
| 2411514042 | Namira Nawar |
| 2411432642 | Seaam Ahmed |

CamScanner

Experiment Name: Design of a 2-bit Logic Unit

Introduction: In this experiment we will construct a 2-bit logic unit which is actually a part of an ALU. This logic unit will have 4 micro-operations which are AND, OR, XOR, and NOT operations. Logic micro operations are very useful for manipulating individual bits or a portion of a word stored in a register. They can be used to change bit values, delete a group of bits or insert a new set of bits in a register. As we are going to design a 2-bit logic unit, we will have two outputs which is one output for each of the 2-bits.

Objective: (i) Understanding the basic concepts and working principle of 2-bit logic unit as a part of ALU.

(ii) Designing and implementing basic logic operations for 2-bit binary inputs.

CS CamScanner

(iii) Analyzing how selection inputs determine the required logic operation at the the output.

## Equipments:

(i) Trainer Board

(ii) IC 7404, 7408, 7432, 7486, 74F153.

(iii) Wires for connection.

**Theory:** A Logic Unit is a fundamental part of an ALU that performs logical operations on binary data. Logical operations are essential in digital systems for decision making, data manipulation and control applications. In this experiment, a 2-bit logic unit is designed to perform basic logic operations on 2-bit binary inputs.

Let the 2-bit inputs be $A=(A_1, A_0)$ and $B=(B_1, B_0)$

The logic unit performs operations bit-by-bit

meaning each bit of A operates with the corresponding bit of B ~~oper~~ independently. The output is also a 2-bit result, denoted as $F = (F_1, F_0)$

The logic operations implemented here are:

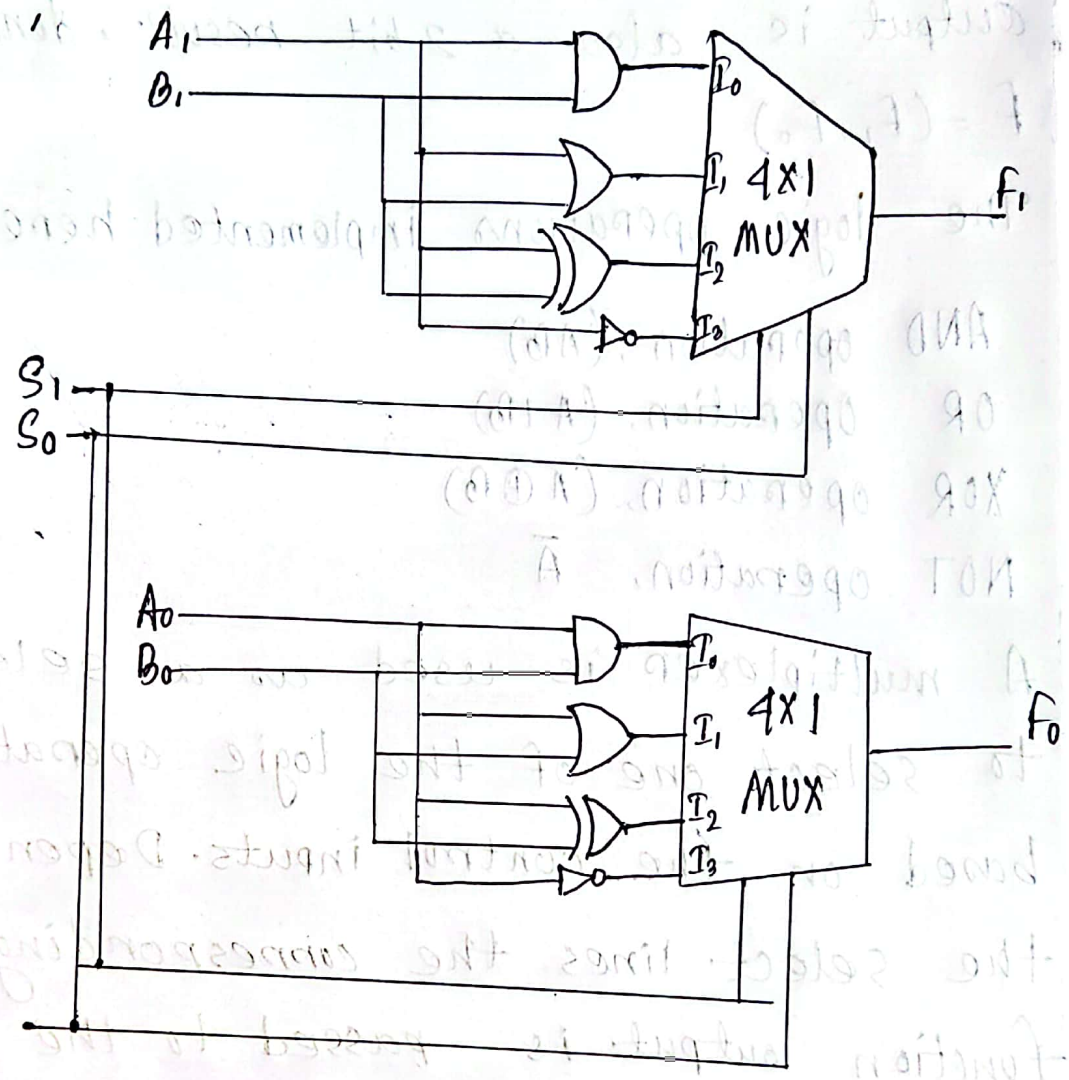
AND operation, $(AB)$
OR operation, $(A+B)$
XOR operation, $(A \oplus B)$
NOT operation, $\bar{A}$

A multiplexer is used as a selection unit to select one of the logic operation outputs based on the control inputs. Depending on the select lines, the corresponding logic function output is passed to the final output. This allows multiple operations to be implemented within a single logic unit.

The 2-bit logic unit therefore produces two outputs ($F_1$ and $F_0$) each corresponding to one bit of the result.

CamScanner

# Logic Diagram:



## Discussion:

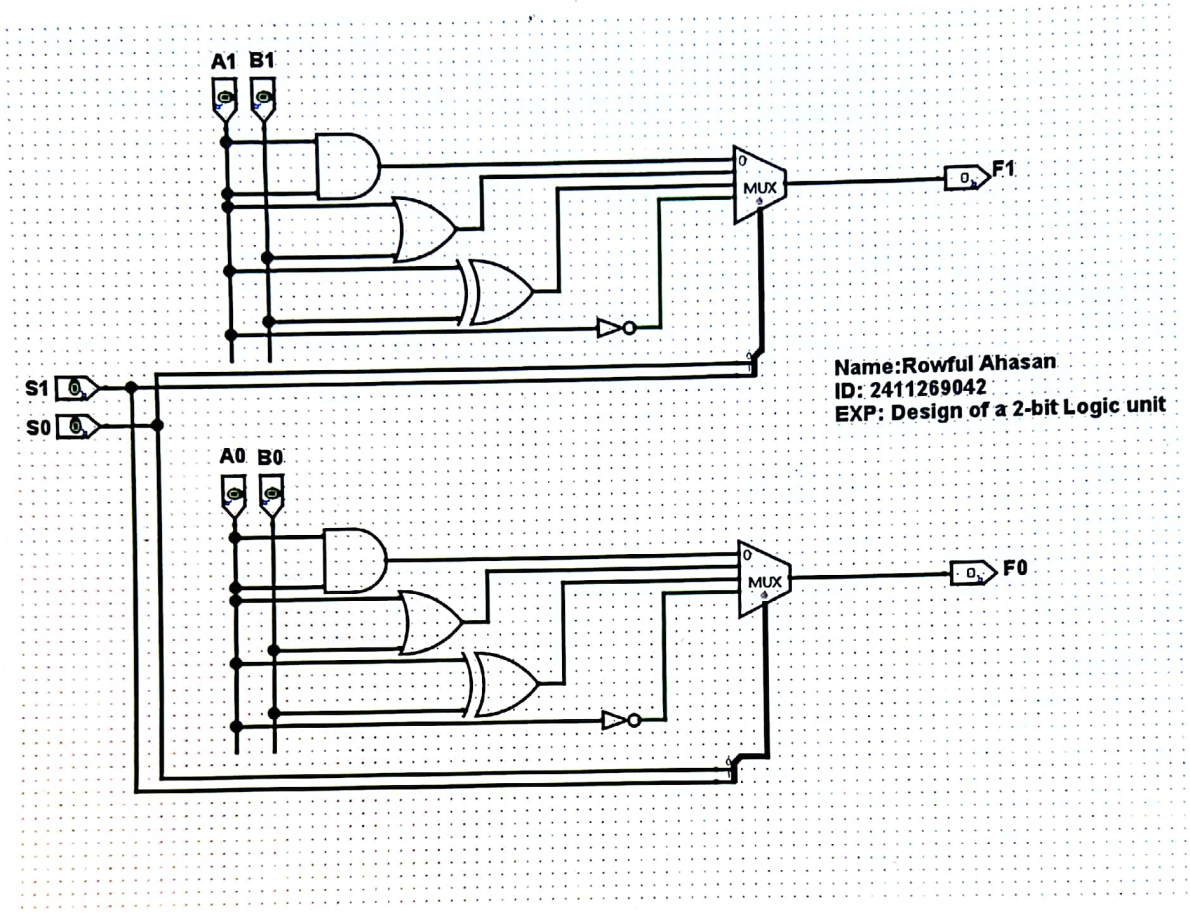
In this experiment we conducted a 2-bit logic unit using AND, OR, XOR and NOT gates and MUX as selection input. During the build on the trainer board, we noticed that our wiring had some issues as the leds were giving wrong outputs. We checked the build again and noticed that instead of inputing A1, A0, B1, B0 respectively, we inputed A1, B1, A0, B0 respectively. We fixed the problem and the logic unit worked properly.

CS CamScanner

## Truth Table:

| A1 | A0 | B1 | B0 | AND1 | AND2 | OR1 | OR2 | XOR1 | XOR2 | NOTA1 | NOTA0 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 |
| 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |

CamScanner


A1
B1
MUX
F1
S1
S0
A0
B0
MUX
F0
Name:Rowful Ahasan
ID: 2411269042
EXP: Design of a 2-bit Logic unit

CamScanner

**Lab Manual**
Department of Electrical and Computer Engineering
School of Engineering and Physical Sciences
North South University, Bashundhara, Dhaka-1229, Bangladesh

Experiment No: 1
Experiment Name: Design of a 2-bit Logic unit.

**Introduction:**

In this experiment you will construct a 2-bit logic unit which is actually a part of an ALU. This logic unit will have 4 micro-operations which are AND, OR, XOR and NOT operations. Logic micro operations are very useful for manipulating individual bits or a portion of a word stored in a register. They can be used to change bit values, delete a group of bits or insert a new set of bits in a register. As we are going to design a 2-bit logic unit, we will have two outputs which is one output for each of the 2 bits.

**Equipments:**

- Trainer board
- IC 7404,7408,7432,7486, 74F153
- Wires for connection.

**Truth Table:**

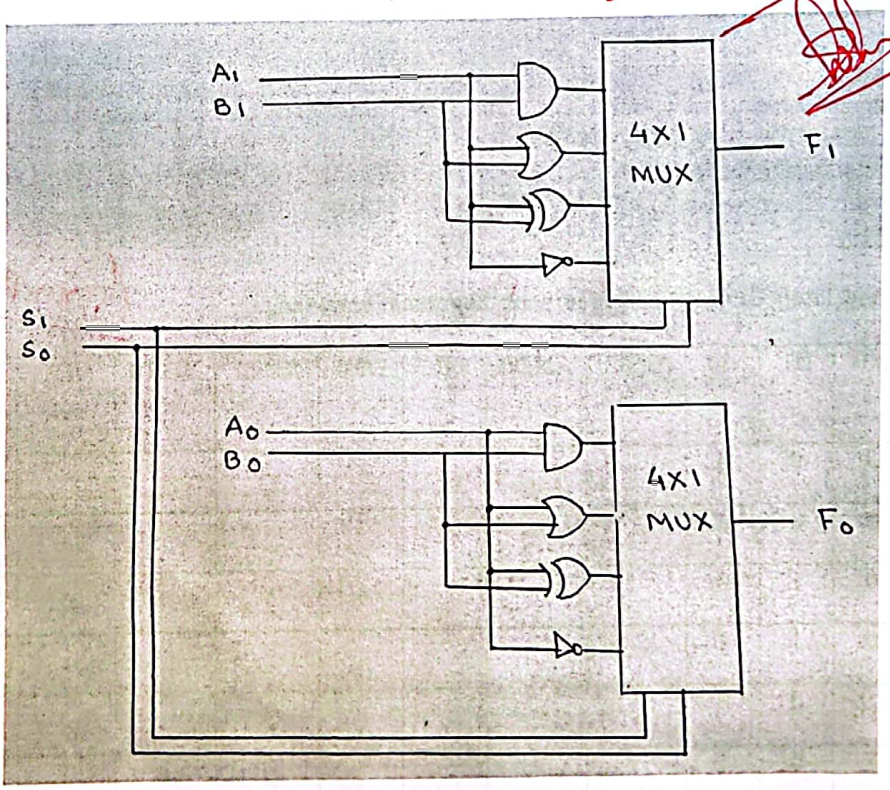
Complete the Truth Table according to your theoretical knowledge.

| A1 | A0 | B1 | B0 | AND1 | AND0 | OR1 | OR0 | XOR1 | XOR0 | NOT A1 | NOT A0 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 |
| 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 |

CamScanner

| A1 | | B1 | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |

**Logic Diagram:**




**Procedure:**